

Crypto Trading Bot

Binance Futures | RSI + MEV Prediction

Developer Guide

Version 1.0 | February 2026

1. Project Overview

This is a cryptocurrency algorithmic trading bot for Binance Futures. It uses two complementary signal sources:

- **RSI (Relative Strength Index)** - Technical momentum indicator on price data
- **MEV Prediction (Optional)** - Mempool monitoring for large pending DEX swaps

The bot connects to Binance Futures API to execute trades, with support for dry-run/paper-trading mode, configurable risk management (stop-loss, take-profit, position sizing), and structured logging for audit trails.

2. Architecture

The project follows a modular architecture with clear separation of concerns:

Core Modules

- `src/main.py` - Entry point with signal handling and graceful shutdown
- `src/bot.py` - `TradingBot` class orchestrating the polling loop
- `src/exchange.py` - Binance Futures API client wrapper
- `src/strategy.py` - RSI and combined RSI+MEV signal evaluation
- `src/indicators.py` - RSI calculation from OHLCV data
- `src/risk_manager.py` - Position sizing, stop-loss, take-profit
- `src/utils.py` - Logging setup and rounding helpers

MEV Prediction Modules

- `src/mempool_monitor.py` - Connects to Ethereum/BSC node, monitors pending transactions
- `src/mev_analyzer.py` - Computes pressure score, whale detection, swap velocity
- `src/dex_abi.py` - DEX router addresses and ABI definitions
- `src/token_price_cache.py` - Cached token-to-USD price lookups

Configuration

- `config/settings.py` - All bot parameters loaded from environment variables
- `.env.example` - Template for required environment variables

3. Project Structure

```
.
+-- config/
|   +-- settings.py          # Bot configuration
+-- src/
|   +-- __init__.py
|   +-- main.py              # Entry point
|   +-- bot.py               # Trading loop
|   +-- exchange.py          # Binance API wrapper
|   +-- strategy.py          # RSI + MEV strategy
|   +-- indicators.py         # RSI calculation
|   +-- risk_manager.py       # Risk management
|   +-- utils.py              # Helpers
|   +-- mempool_monitor.py    # Mempool monitoring
|   +-- mev_analyzer.py       # MEV signal analysis
|   +-- dex_abis.py           # DEX ABIs & addresses
|   +-- token_price_cache.py  # Token price cache
+-- tests/
|   +-- test_indicators.py
|   +-- test_strategy.py
|   +-- test_strategy_mev.py
|   +-- test_risk_manager.py
|   +-- test_mev_analyzer.py
|   +-- test_mempool_monitor.py
|   +-- test_dex_abis.py
+-- logs/
+-- requirements.txt
+-- .env.example
+-- .gitignore
```

4. Setup Instructions

Prerequisites

- Python 3.9 or higher
- pip package manager
- Binance API credentials (testnet recommended for development)
- (Optional) Ethereum/BSC node RPC URL for MEV monitoring

Installation

```
# Clone the repository
git clone <repo-url>
cd crypto-trading-bot-binance-futures-python-rsi-algorithmic-trading

# Create virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

```
# Configure environment variables
cp .env.example .env
# Edit .env with your credentials
```

Running the Bot

```
python -m src.main
```

The bot starts in DRY_RUN mode by default (paper trading, no real orders).

5. Configuration Reference

Binance API

Variable	Description
BINANCE_API_KEY	Your Binance API key
BINANCE_API_SECRET	Your Binance API secret
BINANCE_TESTNET	Set 'true' for testnet (default: true)

Trading Parameters

Parameter	Default / Description
SYMBOL	BTCUSDT
TIMEFRAME	1h
RSI_PERIOD	14 candles
RSI_OVERSOLD	30 (buy signal threshold)
RSI_OVERBOUGHT	70 (sell signal threshold)
LEVERAGE	1x
RISK_PER_TRADE_PCT	1.0% of balance per trade
STOP_LOSS_PCT	2.0% from entry
TAKE_PROFIT_PCT	4.0% from entry
MAX_OPEN_POSITIONS	1
POLL_INTERVAL_SECONDS	60
DRY_RUN	True (paper trading mode)

MEV Monitoring (Optional)

Variable	Default / Description
MEV_ENABLED	false (master toggle)
MEV_CHAIN	ethereum (or 'bsc')
MEV_RPC_URL	WebSocket or HTTP RPC endpoint
MEV_MIN_SWAP_VALUE_USD	50000 (min swap to track)
MEV_WHALE_THRESHOLD_USD	500000 (whale alert level)
MEV_PRESSURE_WEIGHT	0.3 (0.0=RSI only, 1.0=MEV only)
MEV_POLL_INTERVAL_SECONDS	5

6. How the Strategy Works

RSI Strategy (Core)

RSI (Relative Strength Index) measures momentum on a 0-100 scale. The bot uses standard RSI thresholds:

- RSI < 30 (oversold) -> BUY signal
- RSI > 70 (overbought) -> SELL signal
- RSI between 30-70 -> HOLD

Combined RSI + MEV Strategy

When MEV monitoring is enabled, the bot combines RSI with mempool pressure data using a weighted score:

1. RSI is mapped to a -100 to +100 directional score:

$$\text{rsi_score} = -1 * (\text{rsi} - 50) * 2$$

2. The combined score is computed:

$$\text{combined} = (1 - \text{mev_weight}) * \text{rsi_score} + \text{mev_weight} * \text{mev_pressure}$$

3. Signal thresholds on the combined score:

- combined > +40 -> BUY
- combined < -40 -> SELL
- Between -40 and +40 -> HOLD

Example 1: RSI=25 (oversold, rsi_score=+50) with MEV pressure=+60 (buy pressure) at weight=0.3:
combined = $0.7*50 + 0.3*60 = +53$ -> strong BUY.

Example 2: RSI=25 (oversold) but MEV pressure=-80 (sell pressure) at weight=0.3: combined = $0.7*50 + 0.3*(-80) = +11$ -> HOLD. MEV prevented a bad entry.

MEV Mempool Monitor

The mempool monitor connects to an Ethereum or BSC node via WebSocket and watches for pending transactions sent to DEX routers:

- Uniswap V2 Router
- Uniswap V3 SwapRouter
- PancakeSwap Router (BSC)

It decodes swap function calls, estimates USD values, determines if each swap is a buy or sell of tracked assets (WETH, WBTC, WBNB), and computes a net pressure score from -100 (sell pressure) to +100 (buy pressure).

7. Testing

The project includes 55 tests covering all modules:

```
# Run all tests
pytest -v

# Run a specific test file
pytest tests/test_strategy_mev.py -v
```

Test Files

- test_indicators.py - 5 tests (RSI calculation)
- test_strategy.py - 4 tests (RSI signal evaluation)
- test_strategy_mev.py - 8 tests (combined RSI+MEV evaluation)
- test_risk_manager.py - 10 tests (position sizing, SL/TP)
- test_mev_analyzer.py - 13 tests (pressure, whales, velocity)
- test_mempool_monitor.py - 10 tests (monitoring, direction, USD estimation)
- test_dex_abis.py - 5 tests (token resolution, router mapping)

8. Dependencies

Package	Purpose
python-binance	Binance Futures API client
pandas	OHLCV data manipulation
numpy	Numerical computations
ta	Technical analysis indicators
python-dotenv	Environment variable loading
websocket-client	Real-time market data
web3	Ethereum/BSC node interaction (MEV)
pytest	Test framework

9. Security Notes

- NEVER commit .env files or API keys to version control
- NEVER hardcode credentials in source files
- Always use Binance testnet for development (BINANCE_TESTNET=true)
- The bot defaults to DRY_RUN=True (paper trading)
- All sensitive config is loaded from environment variables
- API keys and secrets are stored only in .env (gitignored)

10. Code Conventions

- PEP 8 style guidelines
- Type hints on all function signatures
- logging module for all output (no print statements)
- Descriptive variable names (e.g. rsi_period, not rp)
- Single-purpose functions
- Graceful error handling with retries for API calls